WARNING Access Tokens defined as JWTs should be handled with care. They are *bearer tokens*, meaning that any application can use them in an HTTP request and get access to an OAuth2 Resource Server. Handling the OIDC/ OAuth2 flow in the backend rather than in an SPA provides better security because we don't expose any tokens to the browser. However, there might be other risks to manage, so carefully consider the *trust boundaries* of your system.

In the next section you'll see how to enhance ID Tokens and Access Tokens to propagate information about user roles.

12.1.2 Customizing tokens and propagating user roles

Both ID Tokens and Access Tokens can include different information about the user, which is formatted as *claims* in the JWT. Claims are simple key/value pairs in JSON format. For example, OpenID Connect defines several standard claims to carry information about the user, like `given_name`, `family_name`, `preferred_username`, and `email`.

Access to such claims is controlled through *scopes*, a mechanism provided by OAuth2 to limit what data an OAuth2 Client can access. You can think of scopes as roles assigned to applications rather than to users. In the previous chapter, we used Spring Security to make Edge Service an OAuth2 Client and configured it with the `openid` scope. That scope grants Edge Service access to the authenticated user's identity (provided in the `sub` claim).

Perhaps you have logged in to a third-party website using GitHub or Google (a social login based on OAuth2). If you did, you might have noticed that right after the authentication step, the service prompted you with a second request about what information from your GitHub or Google account you consented to allow the third party to access. That consent functionality is